Experiment 6

Student Name: Riya Kashyap
UID: 25MCA20033
Branch: MCA-UIC
Section/Group: 1
Semester: 2nd
Date of Performance:24/2/2026
Subject Name: Technical Training
Subject Code: 25CAP-652

1. Aim: Learn how to create, query, and manage views in SQL to simplify database queries and provide a layer of abstraction for end-users..

2. Objective

- **Data Abstraction:** To understand how to hide complex table joins and calculations behind a simple virtual table interface.
- **Enhanced Security:** To learn how to restrict user access to sensitive columns by providing views instead of direct table access.
- **Query Simplification:** To master the creation of views that pre-join multiple tables, making reporting easier for non-technical users.
- **View Management:** To understand the syntax for creating, altering, and dropping views, as well as the naming conventions required for efficient data access

3. Theory:

A **View** is essentially a virtual table based on the result-set of an SQL statement. It does not contain data of its own but dynamically pulls data from the underlying "base tables".

- **Simple Views:** Created from a single table without any aggregate functions or grouping. These are often updatable.
- **Complex Views:** Created from multiple tables using JOINs, or including GROUP BY and aggregate functions. These provide a consolidated summary of the database.
- **Security Layer:** In enterprise environments, views are used to grant permissions on specific subsets of data. For example, a "SalaryView" might exclude the "Employee_SSN" or "Home_Address" columns for privacy.
- **Benefits:** They simplify the user experience, ensure data consistency across reports, and reduce the risk of accidental data modification by providing read-only abstractions.

4. Procedure of the Practical:

Creation of Tables and Insertion of Data

```
-- Step0- Creation of Tables and Insertion of Data
CREATE TABLE departments (
   dept_id SERIAL PRIMARY KEY,
   dept_name VARCHAR(50)
);
CREATE TABLE employees (
   emp_id SERIAL PRIMARY KEY,
   emp_name VARCHAR(50),
   salary NUMERIC,
   status VARCHAR(20),
   dept_id INT REFERENCES departments(dept_id)
);
INSERT INTO departments (dept_name)
VALUES ('HR'), ('IT'), ('Finance');
INSERT INTO employees (emp_name, salary, status, dept_id)
VALUES
('Riya', 50000, 'Active', 1),
('Aman', 60000, 'Inactive', 2),
('Karan', 70000, 'Active', 2),
('Neha', 55000, 'Active', 3);
```

Creating a Simple View (Data Filtering)

```
--Step 1- Creating a Simple View (Data Filtering)
CREATE VIEW ActiveEmployees AS
SELECT emp_id, emp_name, salary, dept_id
FROM employees
WHERE status = 'Active';
SELECT * FROM ActiveEmployees;
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	emp_id integer	emp_name character varying (50)	salary numeric	dept_id integer
1	1	Riya	50000	1
2	3	Karan	70000	2
3	4	Neha	55000	3

Creating a View for Joining Multiple Tables

```
--Step 2- Creating a View for Joining Multiple Tables
CREATE VIEW EmployeeDepartmentView AS
SELECT
   e.emp_id,
   e.emp_name,
   e.salary,
   d.dept_name
FROM employees e
JOIN departments d
```

```
ON e.dept_id = d.dept_id;
SELECT * FROM EmployeeDepartmentView;
```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1 of 1

	emp_id integer	emp_name character varying (50)	salary numeric	dept_name character varying (50)
1	1	Riya	50000	HR
2	2	Aman	60000	IT
3	3	Karan	70000	IT
4	4	Neha	55000	Finance

Creating an Advanced Summarization View

```
--Step 3-Creating an Advanced Summarization View
CREATE VIEW DepartmentStats AS
SELECT
   d.dept_name,
   COUNT(e.emp_id) AS total_employees,
   AVG(e.salary) AS average_salary,
   SUM(e.salary) AS total_salary
FROM departments d
LEFT JOIN employees e
ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
SELECT * FROM DepartmentStats;
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	dept_name character varying (50)	total_employees bigint	average_salary numeric	total_salary numeric
1	Finance	1	55000.000000000000	55000
2	IT	2	65000.000000000000	130000
3	HR	1	50000.000000000000	50000

5. Learning Outcome:

- **Abstraction Proficiency:** Students will be able to create and query views to simplify efficient data access and abstraction.
- **Security Implementation:** Students will understand how to use views for data masking and providing restricted access to sensitive information.
- **Syntactic Accuracy:** Students will demonstrate the correct syntax for creating and querying views, ensuring logical clarity in naming conventions.
- **Real-world Application:** Students will be able to design views for practical domains like Library Management Systems or Payroll Systems to demonstrate functionality.